BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 7

Roll no: 53 PRN No: 12414903 Name of Student: **Sara Ansingkar**

Subject Name : Deep Learning

Problem Statement :

Develop an LSTM-based model for time-series forecasting using stock price, weather, or sales datasets.

Algorithm :

Code and Output:

```
import os
import time
import copy

import kagglehub
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models

from torch.utils.data import DataLoader

import pandas as pd
import matplotlib.pyplot as plt
```

```
path = kagglehub.dataset_download(
    "puneet6060/intel-image-classification"
)

print("Path to dataset files:", path)

TRAIN_DIR = os.path.join(
    path,
    "seg_train",
    "seg_train"
)

TEST_DIR = os.path.join(
    path,
    "seg_test",
    "seg_test"
)

print("Training path:", TRAIN_DIR)
print("Testing path:", TEST_DIR)

print("\nTraining classes:")
print(os.listdir(TRAIN_DIR))

print("\nTesting classes:")
print(os.listdir(TEST_DIR))

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

NUM_CLASSES = 6
BATCH_SIZE = 32
EPOCHS = 5
LR = 0.001

print("\nDevice:", device)

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

train_dataset = torchvision.datasets.ImageFolder(
    root=TRAIN_DIR,
    transform=train_transform
)

test_dataset = torchvision.datasets.ImageFolder(
    root=TEST_DIR,
    transform=test_transform
)

print("\nClasses:", train_dataset.classes)
print("Class mapping:", train_dataset.class_to_idx)

print("Number of training images:", len(train_dataset))
print("Number of testing images:", len(test_dataset))

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
```

```
    num_workers=2
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=2
)

alexnet = models.alexnet(
    weights=models.AlexNet_Weights.DEFAULT
)

# Freeze pretrained layers
for param in alexnet.parameters():
    param.requires_grad = False

# Replace final classifier
in_features = alexnet.classifier[6].in_features

alexnet.classifier[6] = nn.Linear(
    in_features,
    NUM_CLASSES
)

alexnet = alexnet.to(device)

vgg16 = models.vgg16(
    weights=models.VGG16_Weights.DEFAULT
)

# Freeze pretrained layers
for param in vgg16.parameters():
    param.requires_grad = False
in_features = vgg16.classifier[6].in_features

vgg16.classifier[6] = nn.Linear(
    in_features,
    NUM_CLASSES
```

```
)

vgg16 = vgg16.to(device)

resnet50 = models.resnet50(
    weights=models.ResNet50_Weights.DEFAULT
)
for param in resnet50.parameters():
    param.requires_grad = False
in_features = resnet50.fc.in_features

resnet50.fc = nn.Linear(
    in_features,
    NUM_CLASSES
)

resnet50 = resnet50.to(device)

efficientnet = models.efficientnet_b0(
    weights=models.EfficientNet_B0_Weights.DEFAULT
)

for param in efficientnet.parameters():
    param.requires_grad = False
in_features = efficientnet.classifier[1].in_features

efficientnet.classifier[1] = nn.Linear(
    in_features,
    NUM_CLASSES
)

efficientnet = efficientnet.to(device)


models_dict = {
    "AlexNet": alexnet,
    "VGG16": vgg16,
    "ResNet50": resnet50,
    "EfficientNetB0": efficientnet
}
```

```
def train_model(model, name):

    model = model.to(device)

    criterion = nn.CrossEntropyLoss()

    optimizer = optim.Adam(
        filter(
            lambda p: p.requires_grad,
            model.parameters()
        ),
        lr=LR
    )

    best_accuracy = 0.0

    best_weights = copy.deepcopy(
        model.state_dict()
    )

    history = []

    start_time = time.time()

    for epoch in range(EPOCHS):
        model.train()

        correct = 0
        total = 0
        running_loss = 0.0

        for images, labels in train_loader:

            images = images.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()

            outputs = model(images)
```

```
    loss = criterion(
        outputs,
        labels
    )

    loss.backward()

    optimizer.step()

    running_loss += loss.item()

    _, predicted = torch.max(
        outputs,
        1
    )

    total += labels.size(0)

    correct += (
        predicted == labels
    ).sum().item()


train_accuracy = (
    100 * correct / total
)

model.eval()

correct = 0
total = 0

with torch.no_grad():

    for images, labels in test_loader:

        images = images.to(device)
        labels = labels.to(device)
```

```
            outputs = model(images)

            _, predicted = torch.max(
                outputs,
                1
            )

            total += labels.size(0)

            correct += (
                predicted == labels
            ).sum().item()


        test_accuracy = (
            100 * correct / total
        )

        history.append(test_accuracy)

        if test_accuracy > best_accuracy:

            best_accuracy = test_accuracy

            best_weights = copy.deepcopy(
                model.state_dict()
            )


        # Print results
        print(
            f"{name} | "
            f"Epoch {epoch + 1}/{EPOCHS} | "
            f"Train Acc: {train_accuracy:.2f}% | "
            f"Test Acc: {test_accuracy:.2f}%"
        )
    model.load_state_dict(
        best_weights
    )
```

```
    training_time = (
        time.time() - start_time
    ) / 60


    return (
        model,
        best_accuracy,
        training_time,
        history
    )



results = {}
histories = {}

print("=" * 60)
print("Training AlexNet")
print("=" * 60)

trained_alexnet, alexnet_accuracy, alexnet_time, alexnet_history = train_model(
    models_dict["AlexNet"],
    "AlexNet"
)

results["AlexNet"] = {
    "Accuracy (%)": alexnet_accuracy,
    "Training Time (min)": alexnet_time
}

histories["AlexNet"] = alexnet_history

models_dict["AlexNet"] = trained_alexnet

print("=" * 60)
print("Training VGG16")
print("=" * 60)

trained_vgg16, vgg16_accuracy, vgg16_time, vgg16_history = train_model(
```

```
    models_dict["VGG16"],
    "VGG16"
)

results["VGG16"] = {
    "Accuracy (%)": vgg16_accuracy,
    "Training Time (min)": vgg16_time
}

histories["VGG16"] = vgg16_history

models_dict["VGG16"] = trained_vgg16


print("=" * 60)
print("Training ResNet50")
print("=" * 60)

trained_resnet50, resnet50_accuracy, resnet50_time, resnet50_history = train_model(
    models_dict["ResNet50"],
    "ResNet50"
)

results["ResNet50"] = {
    "Accuracy (%)": resnet50_accuracy,
    "Training Time (min)": resnet50_time
}

histories["ResNet50"] = resnet50_history

models_dict["ResNet50"] = trained_resnet50

print("=" * 60)
print("Training EfficientNetB0")
print("=" * 60)

trained_efficientnet, efficientnet_accuracy, efficientnet_time, efficientnet_history =
train_model(
    models_dict["EfficientNetB0"],
    "EfficientNetB0"
```

```
)

results["EfficientNetB0"] = {
    "Accuracy (%)": efficientnet_accuracy,
    "Training Time (min)": efficientnet_time
}

histories["EfficientNetB0"] = efficientnet_history

models_dict["EfficientNetB0"] = trained_efficientnet

results_df = pd.DataFrame(results).T

print("\nFinal Results")
print("=" * 60)

print(results_df)


plt.figure(figsize=(8, 5))

plt.bar(
    results_df.index,
    results_df["Accuracy (%)"]
)

plt.xlabel("Model")
plt.ylabel("Accuracy (%)")

plt.title(
    "Transfer Learning Accuracy on Intel Image Classification Dataset"
)

plt.ylim(0, 100)

plt.show()

plt.figure(figsize=(8, 5))

plt.bar(
```

```
    results_df.index,
    results_df["Training Time (min)"]
)

plt.xlabel("Model")
plt.ylabel("Training Time (minutes)")

plt.title(
    "Training Time Comparison"
)

plt.show()
plt.figure(figsize=(10, 6))

for name, history in histories.items():

    epochs = range(
        1,
        len(history) + 1
    )

    plt.plot(
        epochs,
        history,
        marker="o",
        label=name
    )

plt.xlabel("Epoch")
plt.ylabel("Test Accuracy (%)")

plt.title(
    "Test Accuracy Across Epochs"
)

plt.legend()
plt.grid()

plt.show()
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

NUM_CLASSES = 6
BATCH_SIZE = 32
EPOCHS = 5
LR = 0.001

print("Device:", device)

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
Device: cpu
```

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download(
    "puneet6060/intel-image-classification"
)

print("Path to dataset files:", path)
```

```
Using Colab cache for faster access to the 'intel-image-classification' dataset.
```

[6] ✓ 0s

```
import os

print("Dataset path:")
print(path)

print("\nContents of dataset folder:")

for item in os.listdir(path):
    print(item)
```

```
Dataset path:
/kaggle/input/intel-image-classification

Contents of dataset folder:
seg_train
seg_pred
seg_test
```

[7] ✓ 10s

```
for root, dirs, files in os.walk(path):

    level = root.replace(path, "").count(os.sep)
    indent = "    " * level

    print(f"{indent}{os.path.basename(root)}/")
```

```
intel-image-classification/
    seg_train/
        seg_train/
            mountain/
            street/
            buildings/
            sea/
            forest/
            glacier/
    seg_pred/
        seg_pred/
    seg_test/
        seg_test/
            mountain/
            street/
            buildings/
            sea/
            forest/
            glacier/
```

[8] ✓ 0s

```python
TRAIN_DIR = os.path.join(path, "seg_train", "seg_train")
TEST_DIR = os.path.join(path, "seg_test", "seg_test")

print("Training path:", TRAIN_DIR)
print("Testing path:", TEST_DIR)

print("\nTraining classes:")
print(os.listdir(TRAIN_DIR))

print("\nTesting classes:")
print(os.listdir(TEST_DIR))
```

```
Training path: /kaggle/input/intel-image-classification/seg_train/seg_train
Testing path: /kaggle/input/intel-image-classification/seg_test/seg_test

Training classes:
['mountain', 'street', 'buildings', 'sea', 'forest', 'glacier']

Testing classes:
['mountain', 'street', 'buildings', 'sea', 'forest', 'glacier']
```

```python
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
```